

LoRa Exercises

Marcus Pihl, s214918

March 19, 2026

Contents

1	Part A - LoRa P2P using RN2483 and ESP32	2
1.1	Q1 - ESP32 model	2
1.2	Q2 - Hardware UARTs on ESP32	2
1.3	Q3 - Rewritten RX and TX code for ESP32	3
1.4	Q4 - General description of code functionality	9
1.5	Q5 - Testing and observations	10
2	Part B - RN2483 hook-up guide, ESP32 and Cibicom/Loriot	11
2.1	Getting the device connected	11
2.2	OTAA join and transmit	15
2.3	Server-side confirmation	15
2.4	Downlink	15
3	Notable issues	16

1 Part A - LoRa P2P using RN2483 and ESP32

1.1 Q1 - ESP32 model

We first confirmed the ESP32 model using the chip identification sketch. The serial monitor showed that the board uses an ESP32-D0WD-V3 Rev 3, and that it has 2 cores.

```
ESP32 Chip model = ESP32-D0WD-V3 Rev 3
This chip has 2 cores
Chip ID: 14416984
ESP32 Chip model = ESP32-D0WD-V3 Rev 3
This chip has 2 cores
Chip ID: 14416984
ESP32 Chip model = ESP32-D0WD-V3 Rev 3
This chip has 2 cores
Chip ID: 14416984
ESP32 Chip model = ESP32-D0WD-V3 Rev 3
This chip has 2 cores
Chip ID: 14416984
ESP32 Chip model = ESP32-D0WD-V3 Rev 3
This chip has 2 cores
Chip ID: 14416984
ESP32 Chip model = ESP32-D0WD-V3 Rev 3
This chip has 2 cores
Chip ID: 14416984
```

Figure 1: Output

Using this code:

```
void loop() {
  for (int i = 0; i < 17; i = i + 8 ) {
    chipId |= ((ESP.getEfuseMac() >> (40 - i)) & 0xff) << i;
  }

  Serial.printf("ESP32 Chip model = %s Rev %u\n", ESP.getChipModel(), ESP.
    getChipRevision());
  Serial.printf("This chip has %u cores\n", ESP.getChipCores());
  Serial.print("Chip ID: ");
  Serial.println(chipId);

  delay(3000);
}
```

1.2 Q2 - Hardware UARTs on ESP32

The ESP32 has 3 hardware UARTs: UART0, UART1, and UART2. UART0 is normally used for USB communication, code upload, and the serial monitor. For the RN2483 connection we used an additional hardware UART and remapped it to GPIO18 and GPIO19.

1.2 Comparison

Table 1-1. ESP32 Series Comparison

Ordering code ¹	Core	Chip Revision ²	In-Package Flash/PSRAM	Package	VDD_SDIO Voltage
ESP32-D0WD-V3	Dual core	v3.0/v3.1 ⁴	—	QFN 5'S	1.8 V/3.3 V
ESP32-D0WDR2-V3	Dual core	v3.0/v3.1 ⁴	2 MB PSRAM	QFN 5'S	3.3 V
ESP32-U4WDH	Dual core ³	v3.0/v3.1 ⁴	4 MB flash (80 MHz)	QFN 5'S	3.3 V
ESP32-D0WDO6-V3 (NRND)	Dual core	v3.0/v3.1 ⁴	—	QFN 6'6	1.8 V/3.3 V
ESP32-D0WD (NRND)	Dual core	v1.0/v1.1 ⁵	—	QFN 5'S	1.8 V/3.3 V
ESP32-D0WDO6 (NRND)	Dual core	v1.0/v1.1 ⁵	—	QFN 6'6	1.8 V/3.3 V
ESP32-S0WD (NRND)	Single core	v1.0/v1.1 ⁵	—	QFN 5'S	1.8 V/3.3 V

Figure 2: Compare

The table below specifies the RX and TX pins for each of the three UART ports available in ESP32.

UART Port	Rx	Tx	Useable
UART0	GPIO3	GPIO1	Yes
UART1	GPIO9	GPIO10	Yes but requires the reassignment of pins
UART2	GPIO16	GPIO17	Yes

Figure 3: Caption

1.3 Q3 - Rewritten RX and TX code for ESP32

The original code was written around SoftwareSerial, so we rewrote both TX and RX to use HardwareSerial on the ESP32. We also added reset handling for the RN2483 and kept the UART at 57600 baud.

TX code

```
#include <HardwareSerial.h> //ESP32 needs this
#define LED 2 // random pin

HardwareSerial loraSerial(1); //using the ESP32
String str;
const int RST = 23; // reset pin

void lora_autobaud();
void led_on(); // LED work for seeing it works
void led_off();
int wait_for_ok();
void reset_rn2483(); // Reset used here

void setup() {
  Serial.begin(57600); //Baud is important to match serial to the loraserial
  delay(1000); // to give time for serial monitor

  pinMode(LED, OUTPUT); // it works
  led_off();

  pinMode(RST, OUTPUT);
  digitalWrite(RST, HIGH);

  loraSerial.begin(57600, SERIAL_8N1, 18, 19); // The pins we mapped to RX2 and TX2
  loraSerial.setTimeout(1000); Catch up

  Serial.println("Sketch started");
  Serial.println("UART started");

  reset_rn2483();
  Serial.println("RN2483 reset done"); // to make sure no backlog
```

```

// lora_autobaud();

led_on();
delay(1000);
led_off();

Serial.println("Initing LoRa"); //now we set up all the setting according to
the document

str = loraSerial.readStringUntil('\n');
Serial.println(str);
loraSerial.println("sys get ver");
str = loraSerial.readStringUntil('\n');
Serial.println(str);

loraSerial.println("mac pause"); // only p2p not loraWAN
str = loraSerial.readStringUntil('\n');
Serial.println(str);

// loraSerial.println("radio set bt 0.5");
// wait_for_ok();

loraSerial.println("radio set mod lora");
str = loraSerial.readStringUntil('\n');
Serial.println(str);

loraSerial.println("radio set freq 868100000"); // small changes ok, big not
str = loraSerial.readStringUntil('\n');
Serial.println(str);

loraSerial.println("radio set pwr 14");
str = loraSerial.readStringUntil('\n');
Serial.println(str);

loraSerial.println("radio set sf sf12"); // this is for the spread distance
vs speed vs airtime
str = loraSerial.readStringUntil('\n');
Serial.println(str);

loraSerial.println("radio set afcbw 41.7");
str = loraSerial.readStringUntil('\n');
Serial.println(str);

loraSerial.println("radio set rxbw 125");
str = loraSerial.readStringUntil('\n');
Serial.println(str);

// loraSerial.println("radio set bitrate 50000"); //no need for these
// wait_for_ok();

// loraSerial.println("radio set fdev 25000"); // same as above

```

```

// wait_for_ok();

loraSerial.println("radio set prlen 8");
str = loraSerial.readStringUntil('\n');
Serial.println(str);

loraSerial.println("radio set crc on");
str = loraSerial.readStringUntil('\n');
Serial.println(str);

loraSerial.println("radio set iq1 off");
str = loraSerial.readStringUntil('\n');
Serial.println(str);

loraSerial.println("radio set cr 4/5");
str = loraSerial.readStringUntil('\n');
Serial.println(str);

loraSerial.println("radio set wdt 60000"); //disable for continuous reception
, we played around with it, didnt do much
str = loraSerial.readStringUntil('\n');
Serial.println(str);

loraSerial.println("radio set sync 12");
str = loraSerial.readStringUntil('\n');
Serial.println(str);

loraSerial.println("radio set bw 125");
str = loraSerial.readStringUntil('\n');
Serial.println(str);

Serial.println("starting loop"); // done with settigs
}

void loop() {
  led_on();
  loraSerial.println("radio tx 48656C6C6F48656C6C6F48656C6C6F48656C6C6F"); //
this is what we tested, started with "hello" ended up with 255 bytes
  str = loraSerial.readStringUntil('\n');
  Serial.println(str);
  str = loraSerial.readStringUntil('\n');
  Serial.println(str);
  led_off();
  delay(2000); Only send every 2 sec
}

void lora_autobaud() // didnt use
{
  String response = "";
  while (response=="")
  {

```

```

    delay(1000);
    loraSerial.write((uint8_t)0x00);
    loraSerial.write(0x55);
    loraSerial.println();
    loraSerial.println("sys get ver");
    response = loraSerial.readStringUntil('\n');
}
}

/*
 * This function blocks until the word "ok\n" is received on the UART,
 * or until a timeout of 3*5 seconds.
 */
int wait_for_ok()
{
    str = loraSerial.readStringUntil('\n');
    if ( str.indexOf("ok") == 0 ) {
        return 1;
    }
    else return 0;
}

void led_on()
{
    digitalWrite(LED, 1);
}

void led_off()
{
    digitalWrite(LED, 0);
}

void reset_rn2483()
{
    digitalWrite(RST, LOW);
    delay(200);
    digitalWrite(RST, HIGH);
    delay(500);
}

```

RX code

```

#include <HardwareSerial.h> //ESP32
#include <Arduino.h>

HardwareSerial loraSerial(1); //ESP32
String str;
const int RST = 23;
#define LED 2

void toggle_led() { digitalWrite(LED, !digitalRead(LED)); } //confirmations

```

```

void led_on() { digitalWrite(LED, HIGH); }
void led_off() { digitalWrite(LED, LOW); }

void reset_rn2483() { same as TX
    digitalWrite(RST, LOW);
    delay(200);
    digitalWrite(RST, HIGH);
    delay(500);
}

void sendCmd(const char* cmd) {
    loraSerial.println(cmd);
    String r = loraSerial.readStringUntil('\n');
    r.trim();
    Serial.print(cmd);
    Serial.print(" -> ");
    Serial.println(r);
}

void setup() {
    Serial.begin(57600); //important
    delay(1000);

    pinMode(LED, OUTPUT);
    led_off();

    pinMode(RST, OUTPUT);
    digitalWrite(RST, HIGH);

    loraSerial.begin(57600, SERIAL_8N1, 18, 19); // pins in ESP32
    loraSerial.setTimeout(2000); //wait

    Serial.println("Sketch started");
    Serial.println("UART started");

    reset_rn2483();
    Serial.println("RN2483 reset done");

    while (loraSerial.available()) loraSerial.read();

    sendCmd("sys get ver"); // set up same as TX
    sendCmd("mac pause");
    sendCmd("radio set mod lora");
    sendCmd("radio set freq 868100000");
    sendCmd("radio set pwr 14");
    sendCmd("radio set sf sf7");
    sendCmd("radio set afcbw 41.7");
    sendCmd("radio set rxbw 125");
    sendCmd("radio set prlen 8");
    sendCmd("radio set crc on");
    sendCmd("radio set iq off");
}

```

```

    sendCmd("radio set cr 4/5");
    sendCmd("radio set wdt 0");
    sendCmd("radio set sync 12");
    sendCmd("radio set bw 125");
}

void loop() {
    Serial.println("waiting for a message");
    loraSerial.println("radio rx 0");

    String line1 = loraSerial.readStringUntil('\n');
    line1.trim();
    Serial.print("RX first reply: ");
    Serial.println(line1);

    if (line1 != "ok") { //error handling
        Serial.println("radio not going into receive mode");
        delay(1000);
        return;
    }

    String line2 = loraSerial.readStringUntil('\n'); // we read what we get and
    print it
    line2.trim();
    Serial.print("RX second reply: ");
    Serial.println(line2);

    if (line2.startsWith("radio_rx ")) {
        toggle_led();
        Serial.println("Message received!");
        Serial.print("Payload hex: "); // we couldnt do ASCII only HEX
        Serial.println(line2.substring(9)); // only from 9th character
    } else if (line2 == "radio_err") {
        Serial.println("Receive error / timeout"); //error handling
    } else {
        Serial.println("Unexpected response");
    }

    delay(100);
}

```

```
starting loop
ok
radio_tx_ok
ok
radio_tx_ok
ok
radio_tx_ok
ok
radio_tx_ok
ok
radio_tx_ok
ok
radio_tx_ok
ok
```

Figure 4: TX serial output

```
waiting for a message
RX first reply: ok
RX second reply: radio_rx  48656C6C6F
Message received!
Payload hex:  48656C6C6F
waiting for a message
RX first reply: ok
RX second reply: radio_rx  48656C6C6F
Message received!
Payload hex:  48656C6C6F
waiting for a message
RX first reply: ok
RX second reply: radio_rx  48656C6C6F
Message received!
```

Figure 5: RX serial output

At the end we got the setup to look like this:

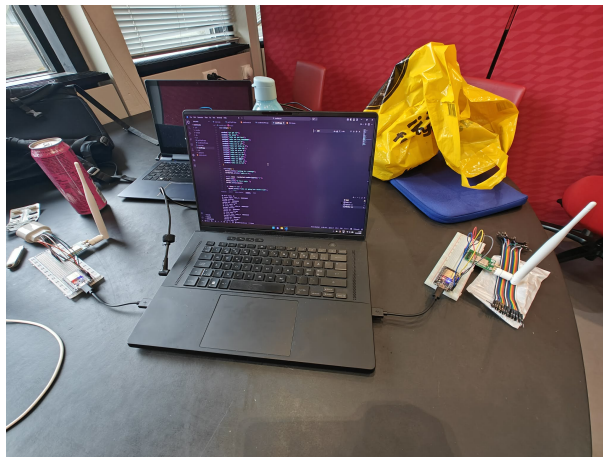


Figure 6: Setup

1.4 Q4 - General description of code functionality

The code first initializes the ESP32 serial connection and the second hardware UART used for the RN2483. After that, the RN2483 is reset and configured through a sequence of radio commands. These commands set the LoRa mode, frequency, transmission power, spreading factor, CRC, coding rate, watchdog timer, sync word, and bandwidth. Once

```
invalid_param
err
invalid_param
err
invalid_param
err
invalid_param
err
invalid_param
err
```

Figure 7: What happens with too big of a load

both boards are configured with the same settings, one board can transmit while the other listens and receives the payload. More details as comments in the code.

1.5 Q5 - Testing and observations

We tested different LoRa settings and message formats. Communication only worked when TX and RX used matching settings such as frequency and spreading factor. HEX payloads worked as expected, while ASCII required attention because the RN2483 handles radio messages in hexadecimal form, we couldn't send ascii so we had to translate at both ends if we wanted. Lower spreading factors gave faster communication, while higher spreading factors are better for range. We also tested larger messages and observed that long payloads eventually fail (around 255 bytes) . 10 bytes: RX first reply: radio_rx 48656C6C6F48656C6C6F then, 20 bytes: RX first reply: radio_rx 48656C6C6F48656C6C6F48656C6C6F48656C6C6F48656C6C6F We tried 145 bytes which worked, and then doubled which failed. we looked up limit and tried to match that. There was no security we were just blasting it on the frequency everyone could see it if they had the same. Since we did it in the library i dont think there was anyone else, but if we did it in class im sure we'd have caught a bunch.

2 Part B - RN2483 hook-up guide, ESP32 and Cibi-com/Loriot

2.1 Getting the device connected

For the LoRaWAN part, we connected the ESP32 to the RN2483 using UART and reset, and then adapted the example code to work on ESP32 with `HardwareSerial`. After startup, the code reads the hardware EUI from the RN2483, which is then used to enroll the device on the Loriot server. We used the following code, which was the examples modified same way as above:

```
#include <rn2xx3.h>
#include <HardwareSerial.h>

HardwareSerial loraSerial(1);
String str;
const int RST = 23;

#define LED 2
#define RESET 23
#define RST 23
#define RXD2 18
#define TXD2 19

//create an instance of the rn2xx3 library,
//giving the software UART as stream to use,
//and using LoRa WAN
rn2xx3 myLora(loraSerial);
void toggle_led() { digitalWrite(LED, !digitalRead(LED)); }
void led_on() { digitalWrite(LED, HIGH); }
void led_off() { digitalWrite(LED, LOW); }

// the setup routine runs once when you press reset:

void initialize_radio()
{
    //reset RN2xx3
    pinMode(RESET, OUTPUT);
    digitalWrite(RESET, LOW);
    delay(100);
    digitalWrite(RESET, HIGH);

    delay(100); //wait for the RN2xx3's startup message
    loraSerial.flush();

    //check communication with radio
    String hweui = myLora.hweui();
    while(hweui.length() != 16)
```

```

{
  Serial.println("Communication with RN2xx3 unsuccessful. Power cycle the
board.");
  Serial.println(hweui);
  delay(10000);
  hweui = myLora.hweui();
}

//print out the HWEUI so that we can register it via ttncctl
Serial.println("When using OTAA, register this DevEUI: ");
Serial.println(hweui);
Serial.println("RN2xx3 firmware version:");
Serial.println(myLora.sysver());

//configure your keys and join the network
Serial.println("Trying to join TTN");
bool join_result = false;

//ABP: initABP(String addr, String AppSKey, String NwkSKey);
//join_result = myLora.initABP("02017201", "8D7FFEF938589D95AAD928C2E2E7E48F
", "AE17E567AECC8787F749A62F5541D522");

//OTAA: initOTAA(String AppEUI, String AppKey);
join_result = myLora.initOTAA("BE7A000000001465", "1
ACEFC79682A1596EB18EC605BB5C415");

while(!join_result)
{
  Serial.println("Unable to join. Are your keys correct, and do you have TTN
coverage?");
  delay(60000); //delay a minute before retry
  join_result = myLora.init();
}
Serial.println("Successfully joined TTN");
}

void setup() {
  // LED pin is GPIO2 which is the ESP8266's built in LED
  pinMode(2, OUTPUT);
  led_on();

  // Open serial communications and wait for port to open:
  Serial.begin(57600);
  loraSerial.setRxBufferSize(1024);
  loraSerial.begin(57600, SERIAL_8N1, RXD2, TXD2);
  loraSerial.setTimeout(1000);

  delay(1000); //wait for the arduino ide's serial console to open

  Serial.println("Startup");
}

```

```

initialize_radio();

//transmit a startup message
myLora.tx("TTN Mapper on ESP8266 node");

led_off();
delay(2000);
}

// the loop routine runs over and over again forever:
void loop() {
    led_on();

    Serial.println("TXing");
    myLora.tx("@"); //one byte, blocking function
    String str = myLora.getRx();
    if(str.length() > 0)
    {
        Serial.print("Received downlink: ");
        Serial.println(str);
    }

    led_off();
    delay(2000);
}

```

With hardware like this:

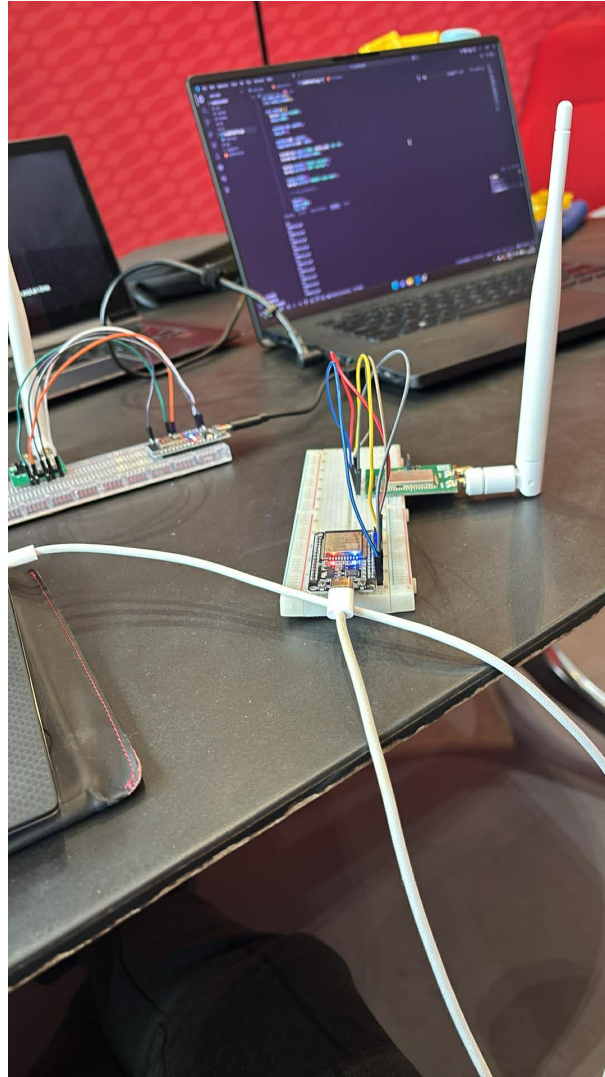


Figure 8: Wires

In the serial monitor we then got:

```
When using OTAA, register this DevEUI:  
0004A30B01107987
```

But suprisingly we had a connect to a network

```
RN2xx3 firmware version:  
RN2483 1.0.5 Oct 31 2018 15:06:52  
Trying to join TTN  
Successfully joined TTN  
TXing  
TXing  
TXing  
TXing  
TXing
```

We didnt know where it was, so we quickly went through next steps.

2.2 OTAA join and transmit

We used OTAA for joining the network. After enrolling the device on Loriot and inserting the AppEUI and AppKey into the code, the module was able to join the network and transmit a small payload periodically. This confirmed that the ESP32 and RN2483 setup worked with the LoRaWAN backend.



Figure 10: Successful join

Figure 9: Lorient enrollment / device page

2.3 Server-side confirmation

On the Lorient side we could confirm both the radio connection and the received payload. The device page showed the latest radio information..

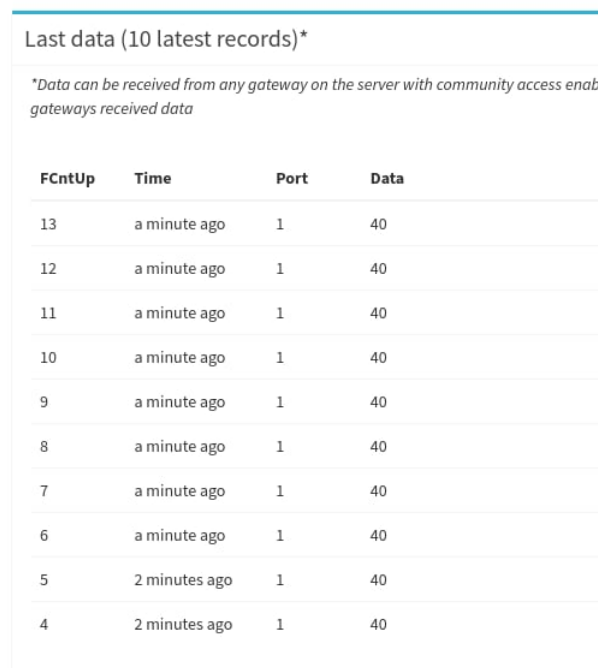


Figure 11: Confirmed gotten

2.4 Downlink

We also prepared the code to read downlink data using `getRx()`. In that way, data queued from the server could be fetched by the ESP32 after an uplink transmission opened the receive window.

Download Date	Payload	Port	Priority	Confirmation	Downloaded
May 15, 2023, 12:00:00 PM	download	1	1		

Figure 12: Queued downlink on Lorient

```
Received downlink: 48656C6C6F
TXing
Received downlink: 48656C6C6F
TXing
Received downlink: 48656C6C6F
TXing
Received downlink: 48656C6C6F
TXing
Received downlink: 48656C6C6F
TXing
Received downlink: 48656C6C6F
TXing
Received downlink: 48656C6C6F
TXing
Received downlink: 48656C6C6F
TXing
Received downlink: 48656C6C6F
TXing
```

Figure 13: Received downlink in serial monitor

3 Notable issues

We had a lot of issues on the original RX, and had to rewrite it from bottom up with a completely different structure, this was done with a lot of research. The issue made it so the TX worked fine, but the RX couldn't find the data at all. We also had some issues matching the serial rates, and discovered how important that is, otherwise the data is unreliable.